Lab 02 · Build y pruebas automatizadas con CodeBuild

Duración estimada: 55 minutos **Sesión:** 1 **Entorno:** GitHub Codespaces con un emulador local de AWS · sin cuenta de AWS

Objetivo

Crear un proyecto de CodeBuild que, a partir de un código fuente en S3, ejecute análisis de estilo y pruebas unitarias en un contenedor, deje los registros en CloudWatch Logs y genere un artefacto en S3. Después romperás una prueba para ver cómo falla el build y cómo la integración continua impide que el cambio avance.

Cómo funciona el entorno

Igual que en el Lab 01: trabajas en un Codespace y la CLI de AWS habla con el emulador en `http://localhost:4566`. Lo interesante de esta práctica es que el emulador **ejecuta de verdad** el build: arranca un contenedor Docker con la imagen del proyecto, copia dentro el código fuente y ejecuta las fases del `buildspec.yml`, igual que CodeBuild.

> **Diferencias con AWS real en esta práctica.** Usamos la imagen pública `python:3.12-slim` en lugar de las imágenes gestionadas de CodeBuild, así que no usamos `runtime-versions`. Los informes de pruebas (`reports`) y la lectura de Parameter Store desde el buildspec (`env: parameter-store`) no están disponibles en el emulador: los resultados de las pruebas los verás en los logs, y al final de la práctica tienes cómo se escribirían en AWS real. No hay consola web: todo se hace con la CLI, que es como se automatiza en la vida real.

Requisitos

- Una cuenta gratuita de GitHub y el Codespace del curso abierto (ver Lab 01). Comprueba que el emulador responde con `aws sts get-caller-identity`.

Punto de partida

La práctica es independiente. En el paso 1 despliegas con CloudFormation un bucket y el rol de servicio que usará CodeBuild.

Paso 1 · Desplegar el punto de partida

```
mkdir -p ~/lab02 && cd ~/lab02

cat > inicio-lab02.yaml <<'EOF2'
AWSTemplateFormatVersion: "2010-09-09"
Description: Lab 02 - punto de partida (bucket y rol de CodeBuild)
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: { "Fn::Sub": "lab02-${AWS::AccountId}" }
      VersioningConfiguration: { Status: Enabled }
  RolCodeBuild:
    Type: AWS::IAM::Role
    Properties:
      RoleName: lab02-codebuild
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
            Principal: { Service: codebuild.amazonaws.com }
            Action: sts:AssumeRole
      Policies:
        - PolicyName: permisos-build
```

```
                  PolicyDocument:
                    Version: "2012-10-17"
                    Statement:
                      - Effect: Allow
                        Action: [logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents]
                        Resource: "*"
                      - Effect: Allow
                        Action: [s3:GetObject, s3:GetObjectVersion, s3:PutObject, s3:GetBucketLocation,
s3:ListBucket]
                        Resource:
                          - { "Fn::GetAtt": [Bucket, Arn] }
                          - { "Fn::Sub": "${Bucket.Arn}/*" }
        Outputs:
          Bucket: { Value: { Ref: Bucket } }
          Rol: { Value: { "Fn::GetAtt": [RolCodeBuild, Arn] } }
        EOF2

        cfn-lint inicio-lab02.yaml && echo "Plantilla correcta"

        aws cloudformation deploy --stack-name lab02-inicio \
          --template-file inicio-lab02.yaml --capabilities CAPABILITY_NAMED_IAM

        export BUCKET=$(aws cloudformation describe-stacks --stack-name lab02-inicio \
          --query "Stacks[0].Outputs[?OutputKey=='Bucket'].OutputValue" --output text)
        export ROL=$(aws cloudformation describe-stacks --stack-name lab02-inicio \
          --query "Stacks[0].Outputs[?OutputKey=='Rol'].OutputValue" --output text)
        echo "Bucket: $BUCKET"; echo "Rol: $ROL"
```

> Fíjate en que el rol solo puede escribir logs y leer y escribir en **su** bucket: mínimo privilegio. El emulador no hace cumplir los permisos de IAM, pero en AWS real un permiso que falte haría fallar el build.

Paso 2 · Aplicación, pruebas y buildspec

```
        mkdir -p app/src app/tests && cd app

        cat > src/calculadora.py <<'EOF2'
        def precio_con_iva(base: float, tipo: float = 0.21) -> float:
            if base < 0:
                raise ValueError("La base no puede ser negativa")
            return round(base * (1 + tipo), 2)


        def descuento(precio: float, porcentaje: float) -> float:
            if not 0 <= porcentaje <= 100:
                raise ValueError("Porcentaje fuera de rango")
            return round(precio * (1 - porcentaje / 100), 2)
        EOF2
        touch src/__init__.py

        cat > tests/test_calculadora.py <<'EOF2'
        import pytest

        from src.calculadora import descuento, precio_con_iva


        def test_iva_general():
            assert precio_con_iva(100) == 121.0


        def test_iva_reducido():
            assert precio_con_iva(100, 0.10) == 110.0


        def test_base_negativa():
```

```
            with pytest.raises(ValueError):
                precio_con_iva(-1)


        def test_descuento():
            assert descuento(200, 25) == 150.0
        EOF2

        cat > requirements-dev.txt <<'EOF2'
        pytest==8.3.3
        ruff==0.6.9
        EOF2

        cat > buildspec.yml <<'EOF2'
        version: 0.2
        env:
          variables:
            ENTORNO: formacion
        phases:
          install:
            commands:
              - pip install --quiet -r requirements-dev.txt
          pre_build:
            commands:
              - echo "Build para el entorno $ENTORNO"
              - ruff check src tests
          build:
            commands:
              - python -m pytest -v --junitxml=reports/junit.xml
          post_build:
            commands:
              - python -m zipfile -c calculadora.zip src
              - echo "Artefacto generado"
        artifacts:
          files:
            - calculadora.zip
        EOF2
```

Comprueba que las pruebas pasan en el Codespace antes de subir nada (buena práctica: la CI no es tu primera línea de pruebas):

```
pip install --quiet -r requirements-dev.txt && python -m pytest -q && ruff check src tests
```

Empaqueta el código fuente y súbelo al bucket:

```
python -m zipfile -c ../fuente.zip src tests requirements-dev.txt buildspec.yml
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
```

Paso 3 · Crear el proyecto de CodeBuild

```
aws codebuild create-project --name lab02 \
  --source type=S3,location=$BUCKET/fuente/fuente.zip \
  --artifacts type=S3,location=$BUCKET,name=artefactos,packaging=NONE \
  --environment type=LINUX_CONTAINER,image=python:3.12-slim,computeType=BUILD_GENERAL1_SMALL \
  --service-role $ROL \
  --logs-config "cloudWatchLogs={status=ENABLED}"

aws codebuild batch-get-projects --names lab02 \
  --query "projects[0].[name,source.type,source.location,environment.image]" --output table
```

Opción	Valor	Para qué sirve
`source`	S3,	De dónde sale el código. En AWS real también puede ser GitHub o CodeCommit.

	`fuente/fuente.zip`	
`environment.image`	`python:3.12-slim`	Contenedor en el que se ejecuta el build. En AWS real, una imagen gestionada como `aws/codebuild/amazonlinux-x86_64-standard:5.0`.
`artifacts`	S3, carpeta `artefactos`	Dónde se deja lo que produce el build.
`service-role`	`lab02-codebuild`	Identidad con la que se ejecuta el build.

Define una pequeña función para lanzar un build, esperar a que termine y enseñar su resultado y sus logs. La usarás varias veces:

```
build() {
  ID=$(aws codebuild start-build --project-name lab02 --query build.id --output text)
  echo "Build lanzado: $ID"
  while true; do
    ESTADO=$(aws codebuild batch-get-builds --ids $ID --query "builds[0].buildStatus" --output text)
    [ "$ESTADO" != "IN_PROGRESS" ] && break
    echo -n "."; sleep 3
  done
  echo; echo "Estado final: $ESTADO"
  aws codebuild batch-get-builds --ids $ID \
    --query "builds[0].phases[].[phaseType,phaseStatus,durationInSeconds]" --output table
  echo "---- Últimas líneas del log ----"
  aws logs tail /aws/codebuild/lab02 --since 10m | tail -40
}
```

Paso 4 · Primer build y revisión

```
build
```

Mientras se ejecuta, abre otra terminal (icono **+** del panel de terminal) y mira qué está pasando por debajo:

```
docker ps
```

Verás, además del emulador, un contenedor `python:3.12-slim` creado para el build: así trabaja CodeBuild, un contenedor efímero por build que se destruye al terminar.

Cuando termine, revisa:

1. En el log, la línea `Build para el entorno formacion` y los cuatro tests `PASSED` de `pytest -v`.
2. En la tabla de fases, cuál ha durado más. Normalmente es `INSTALL`, por la descarga de dependencias.
3. El artefacto:

```
aws s3 ls s3://$BUCKET/artefactos/ --recursive
```

> **Punto de control 1.** Build en `SUCCEEDED`, cuatro pruebas superadas en el log y `calculadora.zip` en el bucket.

Paso 5 · Romper una prueba

Introduce un error en la lógica, vuelve a subir el código y lanza otro build:

```
cd app
sed -i 's/return round(base \* (1 + tipo), 2)/return round(base * tipo, 2)/' src/calculadora.py
python -m zipfile -c ../fuente.zip src tests requirements-dev.txt buildspec.yml
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
build
```

>

> **Punto de control 2.** El build termina en `FAILED`, la fase `BUILD` aparece como fallida, el log muestra dos pruebas `FAILED` con su mensaje (`assert 21.0 == 121.0`) y la fase `POST_BUILD` no genera un artefacto nuevo. Esa es la función de la CI: impedir que un cambio defectuoso avance.

Ahora rompe el estilo en lugar de la lógica y observa en qué fase falla:

```
cd app
sed -i 's/return round(base \* tipo, 2)/return round(base * (1 + tipo), 2)/' src/calculadora.py
echo "import os" >> src/calculadora.py
python -m zipfile -c ../fuente.zip src tests requirements-dev.txt buildspec.yml
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
build
```

`ruff` detecta el import sin usar y el build falla en `PRE_BUILD`, antes de ejecutar las pruebas: falla pronto y barato.

Deja el código en verde:

```
cd app
sed -i '/^import os$/d' src/calculadora.py
python -m zipfile -c ../fuente.zip src tests requirements-dev.txt buildspec.yml
aws s3 cp ../fuente.zip s3://$BUCKET/fuente/fuente.zip
cd ..
build
```

Paso 6 · Cómo sería en AWS real

En una cuenta de AWS, el `buildspec.yml` de esta práctica tendría tres diferencias. Léelas y compáralas con el tuyo:

```
version: 0.2
env:
  parameter-store:                      # lee la configuración de Parameter Store
    ENTORNO: /formacion/entorno
phases:
  install:
    runtime-versions:                   # elige la versión en la imagen gestionada
      python: 3.12
    commands:
      - pip install -r requirements-dev.txt
  # ... pre_build, build y post_build iguales ...
reports:                                # publica el resultado de las pruebas en la consola
  pruebas-calculadora:
    files: [junit.xml]
    base-directory: reports
    file-format: JUNITXML
artifacts:
  files: [calculadora.zip]
cache:                                  # reutiliza las dependencias entre builds
  paths:
    - "/root/.cache/pip/**/*"
```

Reto opcional

Añade a la fase `build` la medición de cobertura con `pytest-cov` (añádelo a `requirements-dev.txt`) y haz que el build falle si la cobertura baja del 90 %: `python -m pytest --cov=src --cov-fail-under=90`. Comprueba que pasa, y después borra una de las pruebas para ver cómo falla.

Preguntas de reflexión

1. ¿Por qué ejecutamos `ruff` en `pre_build` y no después de las pruebas?

2. ¿Qué ventaja tiene leer `ENTORNO` de Parameter Store frente a definirlo como variable en el buildspec?
3. El emulador ejecuta el build en un contenedor igual que CodeBuild. ¿Qué no podrías probar en él y sí necesitarías probar en una cuenta real?

Limpieza

```
aws codebuild delete-project --name lab02
aws s3 rm s3://$BUCKET --recursive
aws cloudformation delete-stack --stack-name lab02-inicio
```

Detén el Codespace al terminar: **Code → Codespaces → ⋯ → Stop codespace**.